ARTIFICIAL INTELLIGENCE LABORATORY

Python Programs — Experiments 6 to 10

Experiments are presented in the requested order: Aim, Program Code, Pseudocode, Input, Output, Result, and Conclusion.

Body text uses a Times New Roman-compatible serif font; program code is monospaced for readability.

Experiments included

6. Vacuum Cleaner Problem (CO3)
7. Breadth-First Search (BFS) (CO3)
8. Depth-First Search (DFS) (CO4)
9. Travelling Salesman Problem (TSP) (CO4)
10. A* Search Algorithm (CO4)

EXPERIMENT 6: Vacuum Cleaner Problem

Course Outcome: CO3

Aim

To write a Python program that models a vacuum-cleaner agent and cleans dirty rooms.

Program Code

```
def vacuum_cleaner(environment, start="A"):
    location = start
    actions = []
    while any(state == "Dirty" for state in environment.values()):
        if environment[location] == "Dirty":
            environment[location] = "Clean"
            actions.append(f"Suck dirt in room {location}")
        else:
            location = "B" if location == "A" else "A"
            actions.append(f"Move to room {location}")
    return actions, environment

rooms = {"A": "Dirty", "B": "Dirty"}
steps, final_state = vacuum_cleaner(rooms)
print("Actions:")
for step in steps:
    print(step)
print("Final environment:", final_state)
```

Pseudocode

```
1. Set the room states and starting location.
2. While any room is dirty:
   a. If the current room is dirty, clean it and record Suck.
   b. Otherwise, move to the other room.
3. Print the actions and final room states.
```

Input

```
Initial environment: A = Dirty, B = Dirty
Starting room: A
```

Output

```
Actions:
Suck dirt in room A
Move to room B
Suck dirt in room B
Final environment: {'A': 'Clean', 'B': 'Clean'}
```

Result

The agent successfully cleans both rooms.

Conclusion

This experiment demonstrates a rule-based agent that performs actions until all rooms are clean.

EXPERIMENT 7: Breadth-First Search (BFS)

Course Outcome: CO3

Aim

To implement BFS and traverse a graph level by level from a starting vertex.

Program Code

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        vertex = queue.popleft()
        order.append(vertex)
        for neighbour in graph.get(vertex, []):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    return order

graph = {
    "A": ["B", "C"], "B": ["D", "E"],
    "C": ["F"], "D": [], "E": ["F"], "F": []
}
print("BFS traversal:", bfs(graph, "A"))
```

Pseudocode

```
1. Mark the start vertex visited and enqueue it.
2. While the queue is not empty:
   a. Remove the front vertex and record it.
   b. Enqueue each unvisited neighbour and mark it visited.
3. Display the traversal order.
```

Input

```
Start vertex: A
Edges: A-B, A-C, B-D, B-E, C-F, E-F
```

Output

```
BFS traversal: ['A', 'B', 'C', 'D', 'E', 'F']
```

Result

BFS visits vertices in breadth-wise order, level by level.

Conclusion

The program uses a queue for BFS. It is useful for level-order traversal and shortest paths in unweighted graphs.

EXPERIMENT 8: Depth-First Search (DFS)

Course Outcome: CO4

Aim

To implement DFS by exploring each branch deeply before backtracking.

Program Code

```
def dfs(graph, start):
    visited = set()
    order = []
    def visit(vertex):
        visited.add(vertex)
        order.append(vertex)
        for neighbour in graph.get(vertex, []):
            if neighbour not in visited:
                visit(neighbour)
    visit(start)
    return order

graph = {
    "A": ["B", "C"], "B": ["D", "E"],
    "C": ["F"], "D": [], "E": ["F"], "F": []
}
print("DFS traversal:", dfs(graph, "A"))
```

Pseudocode

```
1. Create an empty visited set and traversal list.
2. Visit the start vertex and mark it visited.
3. Recursively visit each unvisited neighbour.
4. Continue until all reachable vertices are visited.
5. Display the traversal order.
```

Input

```
Start vertex: A
Edges: A-B, A-C, B-D, B-E, C-F, E-F
```

Output

```
DFS traversal: ['A', 'B', 'D', 'E', 'F', 'C']
```

Result

DFS explores a path deeply before returning to other branches.

Conclusion

The program implements recursive DFS, which is useful in connectivity checks, cycle detection, and path exploration.

EXPERIMENT 9: Travelling Salesman Problem (TSP)

Course Outcome: CO4

Aim

To find the minimum-cost tour visiting every city once and returning to the start, using exhaustive search.

Program Code

```
from itertools import permutations

def travelling_salesman(distance, start=0):
    cities = list(range(len(distance)))
    others = [city for city in cities if city != start]
    best_cost = float("inf")
    best_tour = None
    for route in permutations(others):
        tour = (start,) + route + (start,)
        cost = sum(distance[tour[i]][tour[i + 1]]
                   for i in range(len(tour) - 1))
        if cost < best_cost:
            best_cost, best_tour = cost, tour
    return best_tour, best_cost

distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
tour, cost = travelling_salesman(distance)
print("Best tour:", " -> ".join(map(str, tour)))
print("Minimum cost:", cost)
```

Pseudocode

```
1. Fix a starting city.
2. Generate all permutations of the other cities.
3. Add the starting city to the beginning and end of each tour.
4. Calculate each tour's total distance.
5. Keep the tour with the lowest cost and display it.
```

Input

```
Distance matrix:
[[0, 10, 15, 20],
 [10, 0, 35, 25],
 [15, 35, 0, 30],
 [20, 25, 30, 0]]
Starting city: 0
```

Output

```
Best tour: 0 -> 1 -> 3 -> 2 -> 0
Minimum cost: 80
```

Result

The minimum-cost tour for the given matrix has a total distance of 80.

Conclusion

Exhaustive search finds an optimal tour for small inputs, but its factorial time complexity makes it unsuitable for large instances.

EXPERIMENT 10: A* Search Algorithm

Course Outcome: CO4

Aim

To implement A* search to find a least-cost path using actual path cost and a heuristic estimate.

Program Code

```
import heapq

def a_star(graph, heuristic, start, goal):
    frontier = [(heuristic[start], start)]
    came_from = {start: None}
    g_cost = {start: 0}
    while frontier:
        _, current = heapq.heappop(frontier)
        if current == goal:
            path = []
            node = current
            while node is not None:
                path.append(node)
                node = came_from[node]
            return path[::-1], g_cost[goal]
        for neighbour, edge_cost in graph.get(current, {}).items():
            new_cost = g_cost[current] + edge_cost
            if new_cost < g_cost.get(neighbour, float("inf")):
                g_cost[neighbour] = new_cost
                came_from[neighbour] = current
                priority = new_cost + heuristic[neighbour]
                heapq.heappush(frontier, (priority, neighbour))
    return None, float("inf")

graph = {
    "A": {"B": 1, "C": 4},
    "B": {"C": 2, "D": 5},
    "C": {"D": 1}, "D": {}
}
heuristic = {"A": 4, "B": 3, "C": 1, "D": 0}
path, cost = a_star(graph, heuristic, "A", "D")
print("Best path:", " -> ".join(path) if path else "No path found")
print("Path cost:", cost)
```

Pseudocode

```
1. Set the start node's path cost g to zero.
2. Add it to a priority queue with f = g + h.
3. Remove the node with the smallest f value.
4. If it is the goal, reconstruct and return the path.
5. Otherwise, update neighbours when a cheaper path is found,
   using priority g + h.
6. If the queue empties, report that no path exists.
```

Input

```
Start: A
Goal: D
Edges: A-B=1, A-C=4, B-C=2, B-D=5, C-D=1
Heuristic: h(A)=4, h(B)=3, h(C)=1, h(D)=0
```

Output

```
Best path: A -> B -> C -> D
Path cost: 4
```

Result

A* finds path A → B → C → D with total cost 4.

Conclusion

A* ranks routes using f(n) = g(n) + h(n). With an admissible heuristic, it can find an optimal path.